ECEN 248 - Lab Report

Post Lab #10

A Simple Digital Combination Lock

Section: 524

Ramya Kannan

UIN: 635002011

12/03/2025

TA: Reed Clardy

Objective

The objective of this lab is to design and implement a digital combination lock using a Moore finite state machine (FSM). We will describe the FSM in behavioral Verilog, simulate it in Xilinx Vivado to verify correct operation, and synthesize it for the ZYBO FPGA board. The lock will accept a sequence of password inputs through switches and button presses, and indicate locked/unlocked status through LEDs. We will also modify the design to extend from a three-number combination to a four-number combination.

Design

We first created the combination_lock_fsm module using behavioral Verilog to implement a Moore machine with four states: S0, S1, S2, and S3 using 2-bit state encoding. The reset/"locked" state is S0 and S3 is the unlocked state. From the manual, the three-password combination requires entering: 13, 7, and 9.

The next-state logic was implemented using a case statement inside an always@(*) block. In S0, pressing Key1 (BTN0) with Password = 13 transitions to S1, otherwise it stays in S0. In S1, pressing Key2 (BTN1) with Password = 7 transitions to S2, pressing Key2 with an incorrect password returns to S0, and otherwise it remains in S1. In S2, pressing Key1 (BTN0) with Password = 9 transitions to S3 (unlocked), otherwise it stays in S2. S3 is the unlocked state where the FSM remains until reset. The Lock output was set to 4'b1111 when state equals S3, otherwise 4'b0000.

For the modified four-password design, we expanded the FSM to five states (S0, S1, S2, S3, S4) using 3-bit state encoding. We added state S4 as the new final unlocked state. The first three transitions remained the same, but now S3 is an intermediate state instead of the unlocked state. In S3, pressing Key2 (BTN1) with Password = 1 transitions to S4 (unlocked), pressing Key2 with an incorrect password returns to S0 for security, and otherwise it remains in S3 waiting for input. S4 becomes the final unlocked state. We modified the Lock output logic to activate (4'b1111) only when the state equals S4.

The state storage uses a synchronous positive-edge triggered always block that updates the state register. When Reset is asserted high, the FSM immediately returns to S0 from any state. Otherwise, it updates to nextState on each clock edge. The top-level combination_lock module integrates the FSM with three synchronizer modules that buffer the asynchronous inputs (Key1, Key2, Reset) through three flip-flop stages to prevent metastability when entering the synchronous clock domain.

Results

For the first part of the experiment, the 3-password combination lock FSM was successfully verified and simulated. The testbench console output displayed "All Tests Passed!!!" confirming correct operation. The simulation waveform showed proper state transitions from S0→S1→S2→S3 when the correct passwords (13, 7, 9) were entered with their respective buttons. The Lock output remained 0000 in states S0, S1, and S2, and changed to 1111 in state S3. The design was loaded onto the ZYBO Z7-10 board and the sequence 13→BTN0, 7→BTN1, and 9→BTN0 illuminated all four LEDs indicating it was unlocked.

For the second part of the experiment, we implemented a four-password combination by adding state S4 using the previous code. The state bitsize changed from 2-bit to 3-bits since we are adding another state. The modified FSM now requires the sequence: 13→BTN0, 7→BTN1, 9→BTN0, and 1→BTN1 to

unlock. The Lock output only goes high (1111) when reaching state S4 instead of S3, and the design was successfully synthesized and loaded onto the FPGA. We tested the new password and it correctly unlocked the combination lock with all LED's illuminating.

Conclusion

This lab successfully designed and implemented a digital combination lock using a Moore finite state machine. The 3-password combination lock was verified through simulation and successfully operated on the ZYBO Z7-10 FPGA board, correctly transitioning through states S0→S1→S2→S3 and illuminating all LEDs when unlocked. The FSM properly handled incorrect password entries by returning to the reset state, demonstrating security features. The design was then modified to implement a 4-password combination by adding state S4 and expanding the state encoding from 2-bit to 3-bit. The enhanced design successfully operated on the FPGA, requiring all four passwords in the correct sequence to unlock. This lab demonstrated the practical application of Moore machines for sequential logic design, the use of behavioral Verilog for FSM implementation, and the importance of synchronizers for handling asynchronous inputs.

Post-Lab Deliverables

Figure 1. 3 Combination Lock Code

```
`timescale 1ns / 1ns
`default_nettype none

module combination_lock_fsm (
//outsputs and inputs    output reg [1:0] state,
    output wire [3:0] Lock,      // asserted when unlocked
    input wire Key1,              // unlock button 1
    input wire Key2,              // unlock button 2
    input wire [3:0] Password,    // indicate number
    input wire Reset,             // reset
    input wire Clk                // clock
);
    // state parameter definitions
    parameter S0 = 2'b00,
              S1 = 2'b01,
              S2 = 2'b10,
              S3 = 2'b11;
    //the next state register
    reg [1:0] nextState;
    // next state logic, triggered when key-buttons are pressed
    always @(*) begin
        case (state)
            S0: begin
            //first password entry
                if (Key1 && (Password == 4'd13))
                    nextState = S1;
                else
                    nextState = S0;  //S0
            end
            S1: begin
                //second password empty
                if (Key2 && (Password == 4'd7))
                    nextState = S2;  //password was entered correctly
                else if (Key2 && (Password != 4'd7))
                    nextState = S0;  // wrong password
                else
                    nextState = S1;
            end
            S2: begin
                // third password empty
                if (Key1 && (Password == 4'd9))
                    nextState = S3;  //it unlocks
                else
                    nextState = S2;  // Otherwise, stay in S2
            end
            S3: begin
                // stay unlocked till it resets
                nextState = S3;  // Remain in unlocked state
            end
            default: begin
                //returns to intital stte
                nextState = S0;
            end
        endcase
    end
    //state register: synchronous logic with reset
    always @(posedge Clk) begin
        if (Reset)
            state <= S0;
        else
            state <= nextState;
    end    assign Lock = (state == S3) ? 4'b1111 : 4'b0000;
endmodule
```

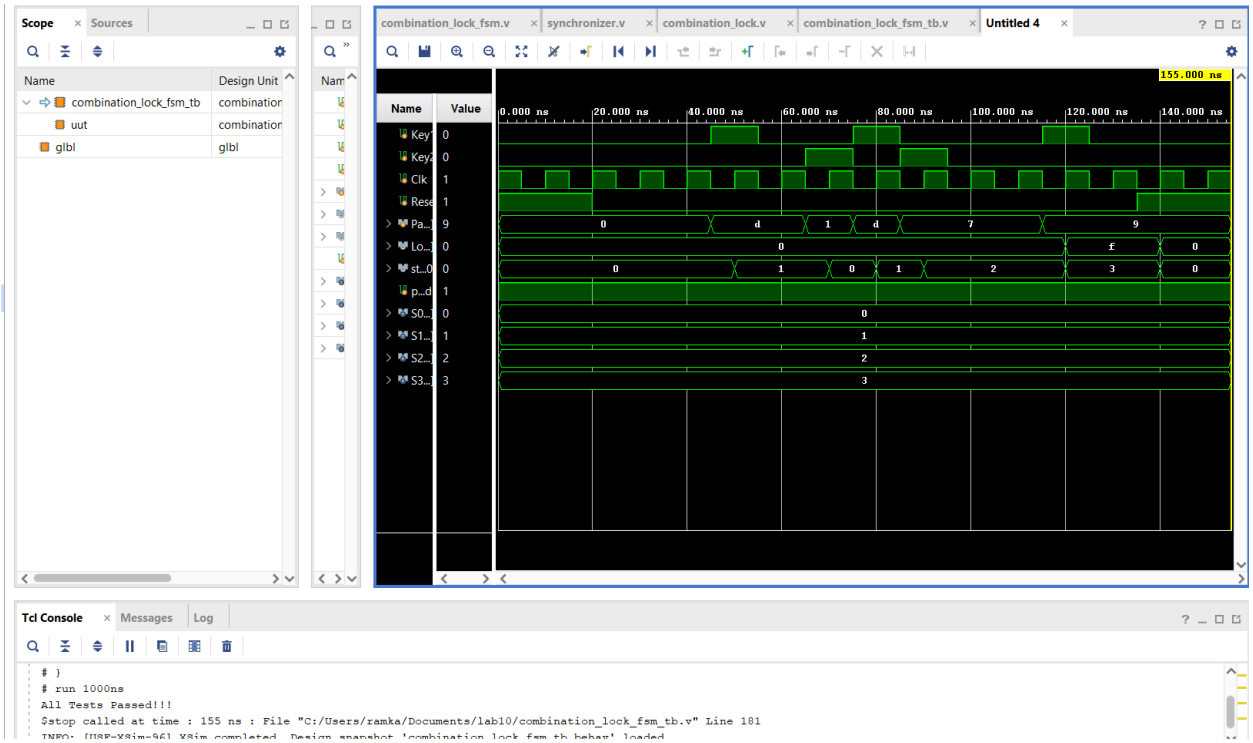
Figure 2. 4 Combination Lock Code

```
`timescale 1ns / 1ns
`default_nettype none

module combination_lock_fsm (
//outsputs and inputs    output reg [1:0] state,
    output wire [3:0] Lock,      // asserted when unlocked
    input wire Key1,              // unlock button 1
    input wire Key2,              // unlock button 2
    input wire [3:0] Password,    // indicate number
    input wire Reset,             // reset
    input wire Clk,
    output reg [2:0] state          // clock
);
    // state parameter definitions
    parameter S0 = 3'b000,
              S1 = 3'b001,
              S2 = 3'b010,
              S3 = 3'b011,
              S4 = 3'b100;
    //the next state register
    reg [2:0] nextState;
    // next state logic, triggered when key-buttons are pressed
    always @(*) begin
        case (state)
            S0: begin
            //first password entry
                if (Key1 && (Password == 4'd13))
                    nextState = S1;
                else
                    nextState = S0;  //S0
            end
            S1: begin
                //second password empty
                if (Key2 && (Password == 4'd7))
                    nextState = S2;  //password was entered correctly
                else if (Key2 && (Password != 4'd7))
                    nextState = S0;  // wrong password
                else
                    nextState = S1;
            end

            S2: begin
                // third password empty
                if (Key1 && (Password == 4'd9))
                    nextState = S3;  //it unlocks
                else
                    nextState = S2;  // Otherwise, stay in S2
            end
            S3: begin
                if(Key2 && (Password == 4'd1))
                    nextState = S4;
                else if(Key2 &&(Password != 4'd1))
                    nextState = S0; //resets
                else
                    nextState = S3; //stays in s3 waitinf for button
                // stay unlocked till it resets
                 // Remain in unlocked state
            end
            S4: begin
                nextState = S4;
            end
        endcase
    end
    //state register: synchronous logic with reset
    always @(posedge Clk) begin
        if (Reset)
            state <= S0;
        else
            state <= nextState;
    end    assign Lock = (state == S4) ? 4'b1111 : 4'b0000;
endmodule
```

Figure 3. Waveform for 3 Lock Combination



1.1e: Likewise, take a look at the simulation waveform and take note of the tests that the test bench performs. Is this an exhaustive test (ie. are all possible cases covered by our test)? Why or why not?

No, this is not an exhaustive test. It only tests for one incorrect password and the reset function. We would need to test all possible password combinations at each state, test incorrect passwords at all states, and cover edge cases with button presses. The current testbench only validates the FSM works for the intended use case and basic error handling.

Student Feedback

1. What did you like most about the lab assignment and why? What did you like least about it and why?

I liked the use of the FPGA board and modifying the locks.

2. Were there any sections of the lab manual that were unclear? If so, what was unclear? Do you have any suggestions for improving the clarity?

No, the manual was clear.

3. What suggestions do you have to improve the overall lab assignment?

No, I have no suggestions for this lab assignment.